

« Les données propres sont rares. Les pipelines robustes sont rares. Celui qui maîtrise les deux est précieux. »

PRINCIPE FONDATEUR :

Le client voit le dashboard. Il ne voit pas les 40 heures passées à comprendre pourquoi une colonne "date" contient des strings, des None et deux formats différents.

Ton travail : rendre l'invisible irréprochable.

SECTION 0 – AVANT DE TOUCHER AUX DONNÉES (Ce que la plupart sautent et regrettent)

0.1 COMPRENDRE LE MÉTIER D'ABORD

- Quel est le processus réel qui génère ces données ?
- Qui les saisit ? Avec quel outil ? Depuis quand ?
- Y a-t-il eu des migrations, des changements de système ?
- Quelles décisions seront prises avec ces données ?

RÈGLE D'OR : Une heure passée à comprendre le métier économise trois heures de débogage.

0.2 CARTOGRAPHIER LES SOURCES AVANT DE CODER

Source	Format	Volume	Fréquence MAJ

0.3 DÉFINIR LA DESTINATION AVANT L'EXTRACTION

- Où vont les données finales ? (BDD, fichier, API, BI)
- Quel format final est attendu ?
- Qui consommera ces données et comment ?
- Quelle fréquence de rafraîchissement ?

0.4 CHECKLIST PRÉ-PROJET

- [] J'ai accès à toutes les sources identifiées
- [] Je connais le schéma cible (même approximatif)
- [] Le livrable final est défini et validé par le client
- [] Un environnement de test est disponible
- [] Les données contiennent-elles des données sensibles ?
- [] Le client a validé les règles métier par écrit

LES 80% – LE TRAVAIL RÉEL EXTRACT · TRANSFORM · LOAD

ÉTAPE 1 – EXTRACT (Extraction)

"Récupérer la donnée telle qu'elle est, pas telle qu'on espère qu'elle soit."

1.1 SOURCES COURANTES & APPROCHE

- FICHIERS PLATS (CSV, Excel, JSON, XML)
- Toujours inspecter avant de charger :
 - Encodage (UTF-8 ? Latin-1 ? Mixte ?)
 - Séparateur réel (virgule, point-virgule, tab ?)
 - Ligne d'en-tête présente ? À quelle ligne ?
 - Lignes parasites en début / fin de fichier ?
 - Lire en chunks si le fichier est volumineux

- Ne jamais supposer que la structure est stable d'un export à l'autre

BASES DE DONNÉES (SQL, PostgreSQL, MySQL, SQLite)

- Demander un accès lecture seule. Toujours.
- Explorer le schéma avant d'écrire des requêtes
- Identifier les clés primaires et étrangères
- Vérifier les index si extraction sur gros volumes
- Requêtes avec LIMIT lors de la phase d'exploration

APIs REST / JSON

- Lire la documentation de pagination en premier
- Tester l'authentification avant tout développement
- Gérer les rate limits (retry avec backoff exponentiel)
- Logger les réponses d'erreur, ne pas les ignorer
- Versionner les endpoints utilisés

SCRAPING WEB

- Vérifier robots.txt et CGU avant tout
- Identifier si une API officielle existe (souvent oui)
- Préférer les données structurées (JSON-LD, microdata)
- Prévoir la fragilité : le DOM change sans prévenir

1.2 QUESTIONS À SE POSER À L'EXTRACTION

- Est-ce un snapshot ou un flux incrémental ?
- Y a-t-il un champ de date de création / modification pour ne récupérer que le delta ?
- Les données sont-elles complètes ou filtrées à la source ?
- Quelle est la fraîcheur maximale acceptable ?

1.3 PIÈGES CLASSIQUES DE L'EXTRACTION

- [] Encodage non détecté → caractères corrompus
- [] Timeout sur grosses extractions → données tronquées
- [] Fuseau horaire non spécifié → décalages temporels
- [] Pagination ignorée → données incomplètes silencieuses
- [] Accès par credentials en dur dans le code → sécurité
- [] Pas de gestion d'erreur → pipeline qui plante en silence

1.4 BONNES PRATIQUES EXTRACTION

- Stocker la donnée brute telle quelle (raw layer) avant toute transformation. Toujours.
- Horodater chaque extraction
- Logger le nombre de lignes extraites à chaque run
- Paramétrer les credentials via variables d'environnement
- Mettre en place une alerte si le volume extrait s'écarte anormalement de la normale

ÉTAPE 2 – TRANSFORM (Transformation)

"C'est ici que vit la vraie complexité. Ne te presse pas."

2.1 AUDIT DE QUALITÉ – À FAIRE EN PREMIER

COMPLÉTUDE

- Taux de valeurs manquantes par colonne
- Colonnes entièrement vides (à supprimer ou signaler)
- Lignes entièrement vides
- Questions à poser : la valeur manquante signifie-t-elle "inconnu", "zéro" ou "non applicable" ?

COHÉRENCE

- Un même concept a-t-il plusieurs représentations ? ("Paris", "paris", "PARIS", "Paris " avec espace)
- Les types sont-ils cohérents dans une colonne ? (nombres stockés en texte, dates en plusieurs formats)
- Les plages de valeurs sont-elles plausibles ? (âge = 200, montant = -99999)

UNICITÉ

- Doublons exacts (toutes colonnes identiques)
- Doublons fonctionnels (même entité, données légèrement différentes selon la source)

→ Quelle est la règle de déduplication métier ?

VALIDITÉ

- Les valeurs respectent-elles les contraintes métier ?
(date de fin < date de début, stock négatif...)
- Les codes/catégories correspondent-ils à un référentiel ?

INTÉGRITÉ RÉFÉRENTIELLE

- Les jointures entre tables produisent-elles des orphelins ? Combien ? Pourquoi ?

2.2 TRANSFORMATIONS STANDARD

NETTOYAGE DES TEXTES

- strip() sur toutes les colonnes texte
- Normaliser la casse selon le besoin
- Supprimer les caractères spéciaux parasites
- Gérer les valeurs nulles textuelles
("N/A", "null", "None", "-", "" → None réel)

DATES & TEMPS

- Identifier tous les formats présents avant de parser
- Choisir un fuseau horaire de référence et s'y tenir
- Créer des colonnes dérivées utiles :
(année, mois, jour, semaine, trimestre, is_weekend)
- Attention aux dates invalides (30 février, etc.)

NUMÉRIQUES

- Identifier séparateur décimal (, ou .)
- Gérer les unités (k€, M€ → normaliser en €)
- Identifier les outliers (IQR, z-score) → décision métier
- Ne jamais remplacer un outlier sans validation client

CATÉGORIELLES

- Construire un référentiel de valeurs attendues
- Mapper les variantes vers la valeur canonique
- Documenter toutes les règles de mapping
- Créer une catégorie "AUTRE" pour l'inattendu,
ne pas silencieusement ignorer

JOINTURES & ENRICHISSEMENT

- Documenter le type de jointure choisi (inner/left/...) et pourquoi
- Vérifier les cardinalités avant (1-1, 1-N, N-N ?)
- Logger le taux de matching des jointures
- Ne jamais faire une jointure sans vérifier les doublons côté droite (risque d'explosion de lignes)

2.3 RÈGLES DE TRANSFORMATION – À DOCUMENTER SYSTÉMATIQUEMENT

Pour chaque règle appliquée, noter :

Colonne	Règle	Justification

- Chaque règle doit être validée par le client ou un expert métier. Jamais une décision unilatérale sur une ambiguïté fonctionnelle.

2.4 PIÈGES CLASSIQUES DE LA TRANSFORMATION

- [] Modifier la donnée brute (toujours travailler en copie)
- [] Décisions implicites non documentées
- [] Jointures qui multiplient les lignes sans s'en rendre compte
- [] Remplacement de NaN par 0 sans validation métier
- [] Perte de lignes silencieuse lors d'un inner join
- [] Règles de nettoyage qui cassent lors d'un nouveau format

3.1 STRATÉGIES DE CHARGEMENT

FULL LOAD (remplacement complet)

- Simple à implémenter
- Adapté aux petits volumes ou données non incrémentales
- Risque : downtime si chargement long
- Toujours faire un backup avant d'écraser

INCREMENTAL LOAD (ajout du delta)

- Requiert un champ de détection du changement (updated_at, id auto-incrémental, hash...)
- Plus complexe mais indispensable sur gros volumes
- Gérer les mises à jour (UPDATE) vs insertions (INSERT)
- Tester le cas "aucune nouvelle donnée"

UPSERT (INSERT + UPDATE si existence)

- Idéal quand les données source peuvent changer
- Nécessite une clé unique fiable
- Vérifier les performances sur grande table

3.2 VALIDATION POST-LOAD

- Comparer le nombre de lignes source vs destination
- Vérifier les agrégats clés (sommés, moyennes) avant et après chargement
- Tester quelques lignes individuelles manuellement
- Valider les contraintes de la table cible (types, NOT NULL, clés uniques)

3.3 PIÈGES CLASSIQUES DU CHARGEMENT

- [] Chargement sans transaction → données partielles en cas d'erreur
- [] Pas de test de la destination avant d'écrire
- [] Droits insuffisants découverts en production
- [] Chargement en production sans test en staging
- [] Absence de rollback en cas d'échec

ÉTAPE 4 – PIPELINE & ORCHESTRATION

"Un ETL qui tourne une fois n'est pas un ETL. C'est un script."

4.1 RENDRE LE PIPELINE ROBUSTE

LOGGING

- Logger le début et la fin de chaque étape
- Logger le nombre de lignes à chaque transition
- Logger les anomalies sans planter (warn vs error)
- Stocker les logs dans un fichier horodaté

GESTION DES ERREURS

- Distinguer erreur fatale (arrêt) et anomalie tolérable
- Retry automatique sur les erreurs réseau
- Alerter (email, Slack) en cas d'échec critique
- Ne jamais laisser un pipeline échouer silencieusement

IDEMPOTENCE

- Le pipeline peut-il tourner deux fois sans corrompre les données ? Si non, c'est un risque majeur.
- Concevoir pour que chaque run soit reproductible

4.2 PLANIFICATION (selon la complexité du projet)

- SIMPLE → cron job (Linux) ou Task Scheduler (Windows)
- MOYEN → Airflow, Prefect, Dagster
- LÉGER → GitHub Actions, script avec schedule
- CLOUD → AWS Lambda + EventBridge, GCP Cloud Functions

Questions à poser avant de choisir :

- Qui va maintenir l'orchestrateur après moi ?
- Le client a-t-il déjà un outil en place ?
- Le volume justifie-t-il la complexité ?

4.3 DOCUMENTATION DU PIPELINE

Toujours livrer avec le code :

- Schéma de flux (même un ASCII art suffit)
- Description de chaque étape en langage naturel
- Liste des dépendances et versions
- Instructions d'installation et d'exécution
- Description des variables d'environnement requises
- Procédure en cas de panne

LES 20% – CE QUE LE CLIENT VOIT

ÉTAPE 5 – ANALYSE & MODÉLISATION

5.1 ANALYSE EXPLORATOIRE (EDA)

- Distribution de chaque variable clé
- Corrélations entre variables
- Évolution temporelle si données de série
- Segmentation naturelle des données
- Formuler 3 à 5 hypothèses métier à confirmer/infirmier
- Partager les findings intermédiaires avec le client avant de modéliser

5.2 CHOIX DU MODÈLE – HIÉRARCHIE DE SIMPLICITÉ

Toujours commencer par le plus simple :

1. Statistiques descriptives + règles métier simples
2. Régression linéaire / logistique
3. Arbres de décision / Random Forest
4. Gradient Boosting (XGBoost, LightGBM)
5. Réseaux de neurones (seulement si justifié)

- Un modèle simple qui s'explique est souvent plus utile qu'un modèle complexe qui performe légèrement mieux mais que personne ne comprend.

5.3 ÉVALUATION DU MODÈLE

- Choisir la métrique en accord avec le métier, pas avec la technique
- Définir une baseline naïve avant d'évaluer
- Tester sur des données que le modèle n'a jamais vues
- Analyser les erreurs, pas seulement le score global
- Valider avec le client : "est-ce que ce niveau d'erreur est acceptable pour vous ?"

5.4 PIÈGES CLASSIQUES DE LA MODÉLISATION

- [] Data leakage (information du futur dans le train)
- [] Overfitting non détecté (perfs train >> test)
- [] Métrique inadaptée au problème métier
- [] Pas de baseline de référence
- [] Classe déséquilibrée ignorée en classification
- [] Interprétation causale d'une corrélation

ÉTAPE 6 – VISUALISATION & LIVRAISON

6.1 PRINCIPES DE VISUALISATION

- Une visualisation = une question = une décision
- Titrer chaque graphique comme un titre de journal ("Les ventes ont baissé de 23% en Q3" pas "Ventes Q3")
- Mettre en couleur ce qui compte, gris le reste
- Éviter les camemberts au-delà de 3 catégories
- Toujours indiquer la source et la date des données

6.2 CHOISIR LE BON OUTIL

- Reporting ponctuel → Jupyter Notebook, Google Slides

Dashboard statique → Matplotlib, Seaborn, Plotly
Dashboard interactif → Streamlit, Metabase, Power BI,
Tableau, Looker Studio (gratuit)
Rapport automatisé → Script Python → PDF / email

Question clé : qui va maintenir ce dashboard ?

→ Si c'est le client seul : outil no-code préférable
→ Si c'est toi en maintenance : outil que tu maîtrises

6.3 CHECKLIST LIVRAISON FINALE

- [] Les données sources sont documentées
- [] Toutes les règles de transformation sont écrites
- [] Le pipeline tourne sans intervention manuelle
- [] La documentation est lisible par un non-développeur
- [] Une formation a été faite avec le client
- [] Les credentials sont transmis de façon sécurisée
- [] Un contact est défini pour la maintenance
- [] La recette (validation finale) est signée

SECTION TRANSVERSALE – QUALITÉ & MAINTENABILITÉ DU CODE

STRUCTURE DE PROJET RECOMMANDÉE

```
projet_client/  
├── data/  
│   ├── raw/           ← données brutes, jamais modifiées  
│   ├── interim/       ← données en cours de traitement  
│   └── processed/     ← données finales propres  
├── notebooks/         ← exploration et prototypage  
├── src/  
│   ├── extract.py  
│   ├── transform.py  
│   └── load.py  
├── tests/             ← tests unitaires  
├── logs/  
├── .env.example       ← template variables d'environnement  
├── requirements.txt  
└── README.md
```

RÈGLES DE CODE POUR PIPELINE DE PRODUCTION

- Pas de valeurs en dur dans le code (chemins, credentials)
- Chaque fonction fait une seule chose
- Nommer les variables pour ce qu'elles contiennent,
pas pour comment elles sont calculées
- Un test par règle de transformation critique
- Committer à chaque étape qui fonctionne (git)
- Ne jamais pousser des données sur GitHub

CHECKLIST UNIVERSELLE – À RELIRE AVANT CHAQUE PROJET

AVANT

- [] Le livrable final est défini et validé par le client
- [] Les accès à toutes les sources sont opérationnels
- [] Un environnement de test séparé de la production existe
- [] Les règles métier ambiguës ont été clarifiées par écrit

PENDANT

- [] La donnée brute est toujours conservée intacte
- [] Chaque décision de transformation est documentée
- [] Le client valide les findings intermédiaires
- [] Les anomalies sont signalées, pas silencieusement corrigées

APRÈS

- [] Le nombre de lignes source = destination (ou écart justifié)
- [] Le pipeline tourne de bout en bout sans erreur
- [] La documentation est à jour
- [] La formation client a été faite
- [] La recette est signée

LES VÉRITÉS DU TERRAIN SUR L'ETL

- Les données ne seront jamais parfaites. Livre quand même.
 - La moitié des bugs viennent d'un changement silencieux dans les données source. Monitore toujours le volume et la structure des inputs.
 - Ce qui prend 1h en exploration prend 1 semaine en production. Toujours multiplier par 3 l'estimation.
 - "Ça marche sur mon poste" n'est pas une livraison.
 - Le client oubliera que les données étaient un désastre. Il se souviendra que tu as livré à temps.
 - Un pipeline bien documenté que tu peux reprendre 6 mois plus tard vaut 10x un pipeline brillant que personne ne comprend.
-